

Predicting the Dynamics of Microcantilevers for Radiative Heat and Momentum Transfer on the Nanoscale — Snap-in

MIT Mechanical Engineering Department - META Lab

Term: Summer 2024

Name: M. Subhi Abo Rdan (msubhi_a@mit.edu)

Faculty Supervisor: Svetlana V. Boriskina

Direct Supervisor: Simo Pajovic

Overview

In the field of radiative heat transfer, the near-field regime occurs when the sizes of objects and/or the distances between them are smaller than the peak wavelength of thermal radiation, according to Wien's law. The deviation from traditional laws of thermal radiation in the near-field is well-documented; however, the transition to the contact regime—from radiation to conduction—is poorly understood. This transition is often studied using scanning probe microscopy, which includes various techniques that utilize microcantilevers as probes, such as atomic force microscopy (AFM). A significant challenge in these studies is overcoming “snap-in” — the abrupt jump to contact between two closely spaced surfaces due to surface forces. A cantilever with a finite stiffness k approaches a sample surface. When the surface force gradient ∇P exceeds the cantilever's stiffness k , the cantilever beam becomes mechanically unstable, causing it to rapidly deflect towards the surface until contact is made. As shown in the figure:

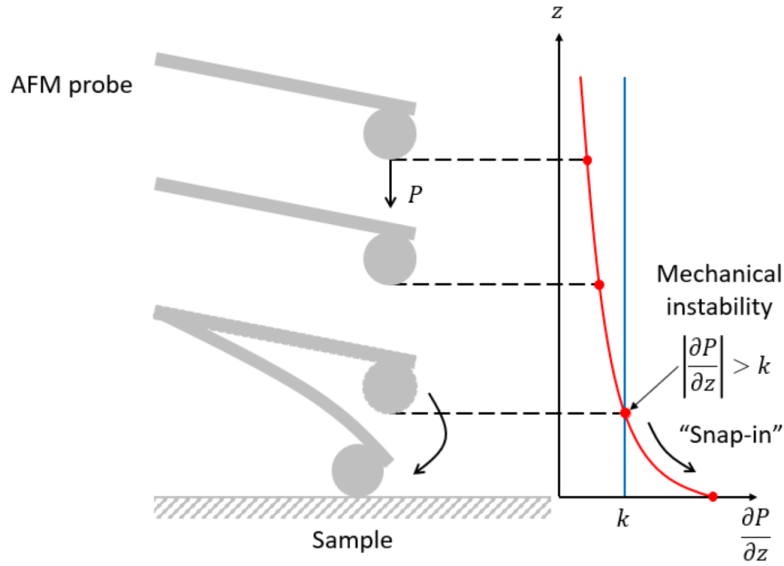


Figure 1: Illustration of snap-in. As the AFM probe moves towards the sample surface, the gradient of the force increases, when it becomes larger than beam stiffness, a mechanical instability will cause snap-in.

In our work, we model the snap-in phenomenon using the Euler-Bernoulli beam equation and build a simulation based on this model by solving the equation numerically using the Crank–Nicolson Finite Difference Method (FDM). We provide estimates for the snap-in dynamics in different AFM experiment setups, specifically the snap-in time and the snap-in distance, defined as the time and distance from when the beam loses stability until it makes contact with the sample.

Mathematical Model

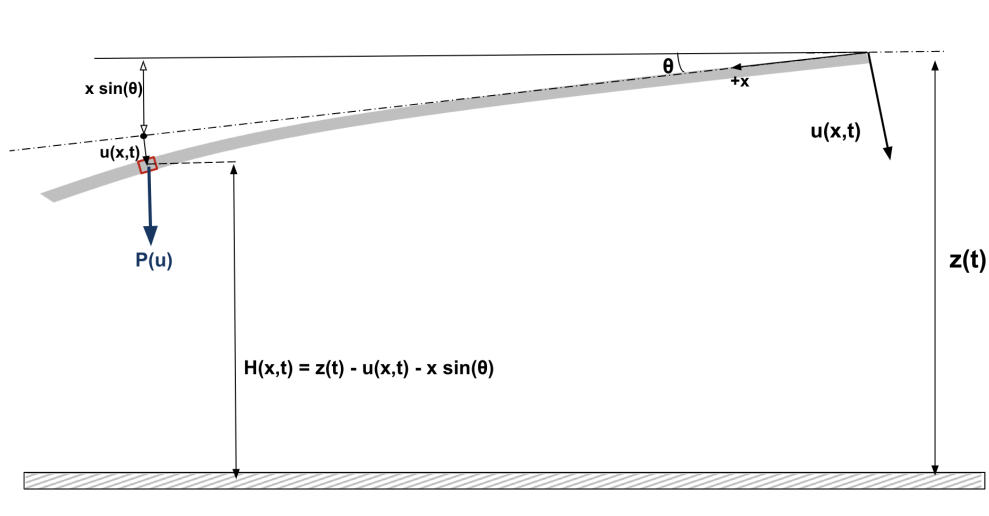


Figure 2: The model of the cantilever with the refined term $x \sin(\theta)$ shown

We seek an implicit finite difference method (FDM) scheme to numerically solve the nonhomogeneous, nonlinear 4th-order partial differential equation:

$$EI \frac{\partial^4 u}{\partial^4 x} = -\rho \frac{\partial^2 u}{\partial^2 t} + \frac{Ab \cos(\theta)}{24\pi(z(t) - u - x \sin(\theta))}$$

We can non-dimensionalize the equation by making the following substitutions:

$$u \rightarrow \nu = u/l$$

$$x \rightarrow \chi = x/L$$

$$t \rightarrow \tau = t/\omega_0^{-1}$$

With letting $\zeta(\tau) = z(\tau)/l$, $\alpha = \frac{L^4 \omega_0^2 \rho}{EI}$, $\beta = \frac{L^4 Ab \cos(\theta)}{24\pi l^4 EI}$ we can have:

$$\Rightarrow \frac{\partial^4 \nu}{\partial^4 \chi} = -\alpha \frac{\partial^2 \nu}{\partial^2 \tau} + \beta \frac{1}{(\zeta - \nu - \frac{L}{l} \sin(\theta) \chi)^3}$$

Our derivation of the scheme to numerically solve the equation. Using the implicit finite difference method, the governing equation of our scheme at a grid point (i,j):

$$\begin{aligned} & \frac{\lambda}{(\Delta \chi)^4} (\nu_{i-2,j+1} - 4\nu_{i-1,j+1} + 2\nu_{i+2,j+1}) + \frac{1-\lambda}{(\Delta \chi)^4} (\nu_{i-2,j-1} - 4\nu_{i-1,j-1} + 2\nu_{i+2,j-1}) = \\ & -\alpha \frac{\nu_{i,j-1} - 2\nu_{i,j} + \nu_{i,j+1}}{(\Delta \tau)^2} + \beta \frac{1}{(\zeta(\tau) - \nu_{i,j} - \frac{L}{l} \sin(\theta) \chi)^3} \end{aligned}$$

Simulation Code

- **cantilever.py** contains the class for the cantilever object, which represents an Atomic Force Microscope (AFM) microcantilever with its physical properties.

Cantilever Class Represents a rectangularly-shaped AFM cantilever. The class includes constructor that initializes the cantilever object with the following properties:

- **volume_density**: Volume density of the cantilever material [kg/m³].
- **natural_frequency**: Natural frequency of the cantilever [Hz].
- **stiffness**: Stiffness of the cantilever [N/m].
- **beam_depth**: Depth of the cantilever beam [m].
- **beam_thickness**: Thickness of the cantilever beam [m].
- **beam_length**: Length of the cantilever beam [m].
- **tip**: Optional parameter representing the tip of the cantilever.
- **tip_distance**: Optional distance from the end of the cantilever to the tip [m].

The constructor calculates additional properties:

- **second_moment_area**: Second moment of area of the beam cross-section.
- **linear_density**: Linear density of the beam.
- **young_modulus**: Young's modulus of the material, calculated using:

$$E = \frac{k \cdot L^3}{3 \cdot I}$$

where k is the stiffness, L is the beam length, and I is the second moment of area.

Tip Class Represents spherical tips used in AFM cantilevers. The class includes a constructor that initializes the tip object with **diameter**: Diameter of the spherical tip [m].

- **fdm.py** contains the implementation of the mathematical numerical methods used in the simulation, highlighted by the FDM functions that solve the Euler-Bernoulli beam equation in both dynamic and static cases.

dynamic_beam_fdm This function solves the partial differential equation (PDE) for the beam using the Finite Difference Method (FDM). The equation is given by:

$$\frac{\partial^4 \nu}{\partial \chi^4} = -\alpha \frac{\partial^2 \nu}{\partial \tau^2} + \frac{\beta}{(\zeta - \nu - \kappa \chi)^3}$$

with boundary conditions:

$$\nu(0) = 0, \quad \nu'(0) = 0, \quad \nu''(1) = 0, \quad \nu'''(1) = 0$$

Parameters include spatial and temporal grid sizes, the maximum number of time nodes, and various constants like α , β , ζ_0 , ν , and κ .

static_beam_bvm This function addresses the boundary value problem (BVP) for the beam under static conditions. The equation is:

$$\frac{\partial^4 \nu}{\partial \chi^4} = \frac{\beta}{(\zeta - \nu - \kappa \chi)^3}$$

with boundary conditions:

$$\nu(0) = 0, \quad \nu'(0) = 0, \quad \nu''(1) = 0, \quad \nu'''(1) = 0$$

The function returns an array representing the initial deflection of the beam, calculated over the spatial grid.

- **simulation.py** contains the simulation code that builds on the mathematical utility functions implemented in 'fdm.py'. It is instantiated with initial conditions from AFM experiments and provides attributes to generate initial beam deflection, run the simulation, compute the stress along the beam, and determine the snap-in time. Additionally, it offers functions to plot the cantilever tip height as a function of time during the snap-in, plot beam deflection, and plot initial deflection.

AFMSimulation Class This class simulates the behavior of AFM cantilevers by solving the beam equation numerically. The main features include:

- **Constructor**

Initializes the simulation with parameters including the cantilever object, surface Hamaker constant, initial height, approach velocity, inclination angle, and grid sizes (`delta_chi` and `delta_tau`).

- **get_theoretical_snap_distance**

Computes the theoretical snap-in distance using the formula:

$$\delta_0 = \left(\frac{Ab \cos(\theta)}{8\pi k} \right)^{1/4}$$

- **get_alpha**

Calculates the dimensionless parameter α in the beam equation:

$$\alpha = \frac{L^4 \omega_0^2 \rho}{EI}$$

- **get_beta**

Computes the ratio of bulk-surface force to restoring force, β , defined as:

$$\beta = \frac{L^4 Ab \cos(\theta)}{24\pi l^4 EI}$$

- **generate_initial_deflection**

Generates the initial deflection of the beam by using the `static_beam_bvp` function to solve for the static case.

- **run**

Runs the simulation using the dynamic beam FDM function, computing the results for beam deflection over time using `dynamic_beam_fdm`.

- **Compute Stress**

Estimates the stress across the cantilever body at the end of the simulation. Optionally plots the stress distribution.

- **Get Snap-In Time**

- **Plot Tip Height**

- **Plot Beam Deflection**

- **Plot Initial Deflection**

Using the simulation

The following script **user.py** demonstrates how to use the simulation by initializing the cantilever object and setting up the simulation environment.

```
import numpy as np
from simulation import AFMSimulation
from cantilever import Cantilever, Tip

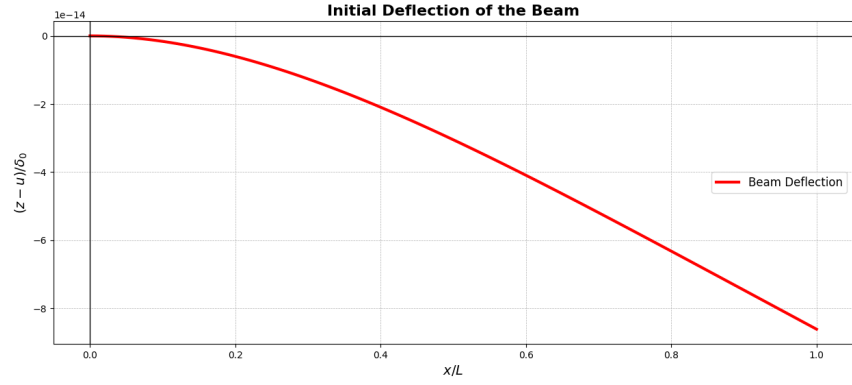
##### Define the used tip #####
my_tip = Tip(diameter= 0)

##### Define the used cantilever #####
my_cantilever = Cantilever( volume_density    = 3170,      #[kg/m^3]
                           natural_frequency = 350e3,     #[Hz]
                           stiffness         = 42,        #[N/m]
                           beam_depth        = 30e-6,     #[m]
                           beam_thickness    = 7.5e-6,    #[m]
                           beam_length       = 1e-6,      #[m]
                           tip               = my_tip,
                           tip_distance      = 0)

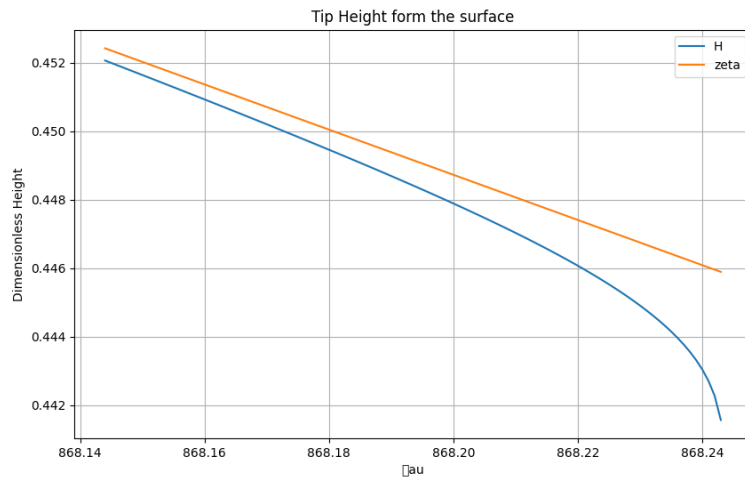
##### Set up the simulation #####
my_simulation = AFMSimulation( cantilever          = my_cantilever,
                               surface_Hamaker_constant = 16.70e-20,    #[J]
                               initial_height        = 2.5e-5,          #[m]
                               approach_velocity     = 1e-2,            #[m/s]
                               inclination_angle      = np.pi*(11/180),  #[rad]
                               delta_chi              = 0.01,
                               delta_tau              = 1e-3,
                               n                      = 1e6)

my_simulation.plot_initial_deflection()
my_simulation.run()
my_simulation.plot_tip_height()
my_simulation.plot_beam_deflection()
print(my_simulation.get_snap_in_time())
```

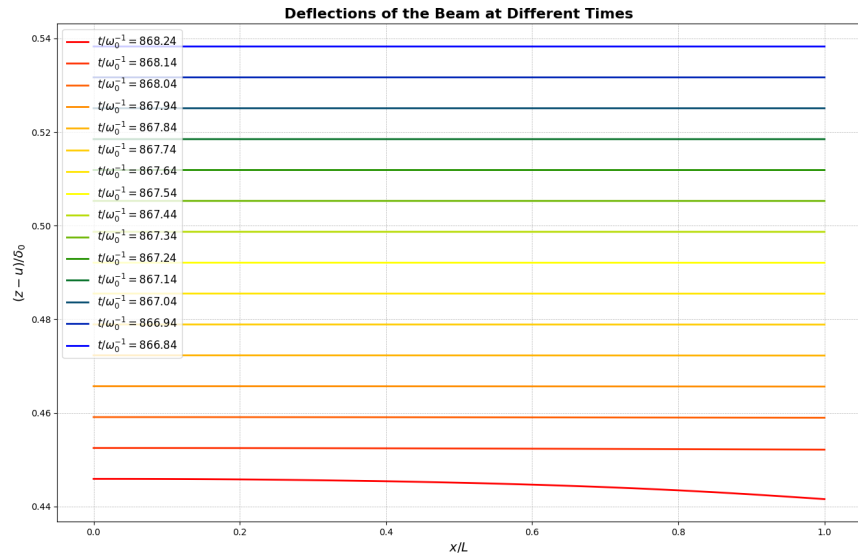
The following are the results of running the above script:



(a) Initial Deflection



(b) Tip Height



(c) Beam Deflection

Figure 3: Results from the AFM simulation example script above.